TERRAFORM

COMPLETE CHEAT SHEET

Intro • Architecture • Commands • Demo Project

Written by AwaisUmarHayat
🔗 Follow for more on LinkedIn: linkedin.com/in/awaisumarhayat



HashiCorp Terraform

01 WHAT IS TERRAFORM?

Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp. It lets you define, provision, and manage cloud infrastructure using a declarative configuration language called HCL (HashiCorp Configuration Language). Write once, deploy anywhere.

• Why Terraform?

Feature	Description
Multi-Cloud	AWS, Azure, GCP, and 1000+ providers via plugins
Declarative	You declare what you want — Terraform figures out how
Idempotent	Run multiple times — same result every time
Version Control	Infrastructure code stored in Git like application code
State Management	Tracks real-world resources in a state file
Open Source	Free to use, large community & ecosystem

• Terraform vs. Other IaC Tools

Tool	Language	Best For
Terraform	HCL / JSON	Multi-cloud, general IaC
Ansible	YAML	Configuration management
CloudFormation	JSON / YAML	AWS-only infrastructure
Pulumi	Python/JS/Go	Developers preferring real languages
CDK	TypeScript/Python	AWS with programming constructs

• Terraform Workflow Overview

```
# The 4-step Terraform lifecycle:

  Write → Init → Plan → Apply

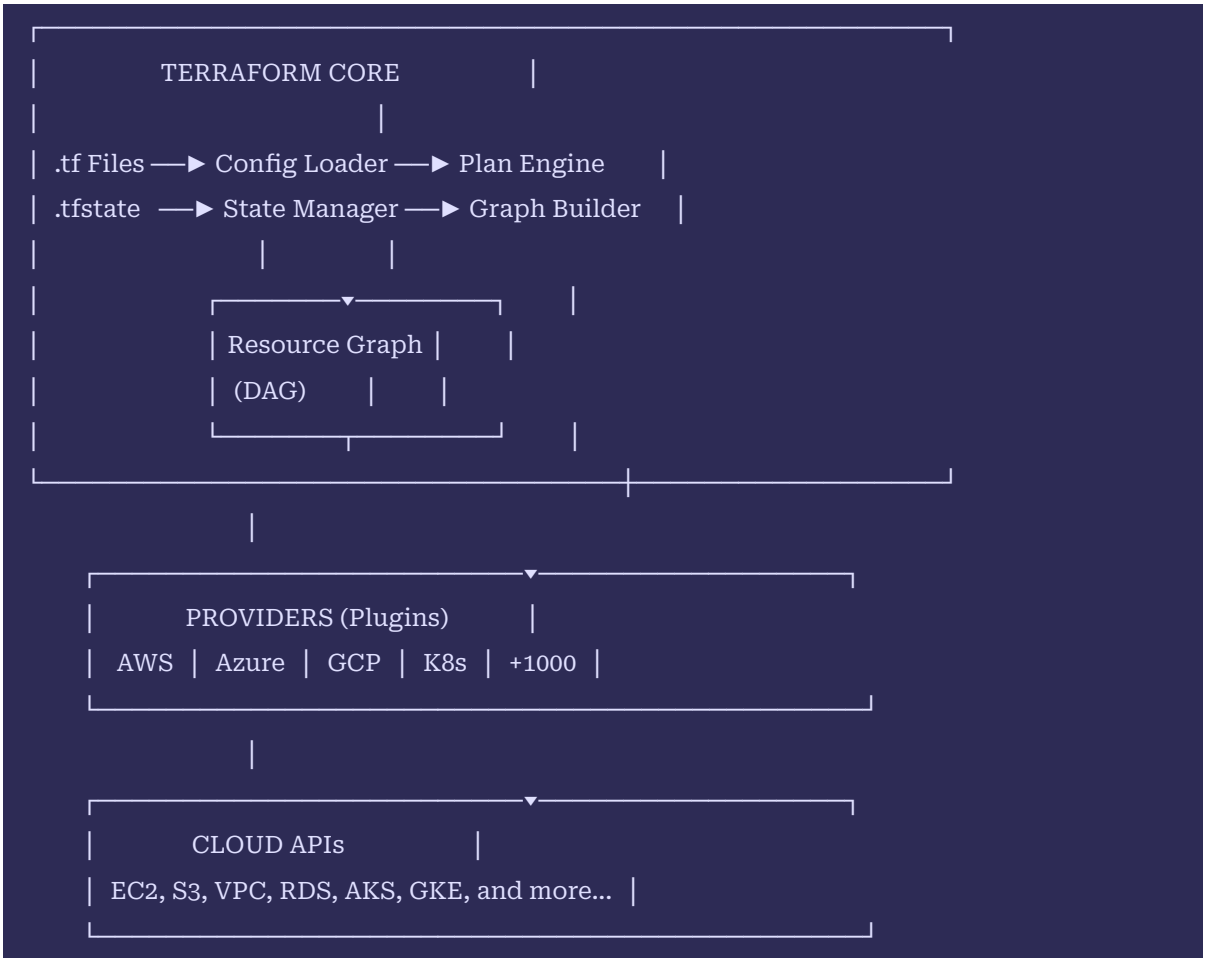
  Write:   Define infrastructure in .tf files (HCL)
  Init:    Download providers & initialize backend
  Plan:    Preview changes before applying
  Apply:   Create/update/destroy real infrastructure
```

02 TERRAFORM ARCHITECTURE

• Core Components

Component	Role
Terraform Core	Reads config & state, creates execution plan, calls providers
Providers	Plugins that talk to APIs (AWS, Azure, GCP, etc.)
Resources	The infrastructure objects managed by Terraform
State File	JSON file tracking what Terraform has deployed
Backend	Where state is stored (local, S3, Terraform Cloud)
Modules	Reusable groups of resources (like functions in code)
Data Sources	Read-only info fetched from providers
Variables	Input parameters to make configs reusable
Outputs	Values exported after apply (IPs, IDs, ARNs)

• How Terraform Works Internally



• State File — The Brain of Terraform

terraform.tfstate is a JSON file that maps your config to real-world resources. Terraform compares desired state (config) vs current state (tfstate) to determine what changes to make. NEVER manually edit this file!

State Storage	Use Case
Local (default)	Learning, solo dev — stores in working directory
Amazon S3 + DynamoDB	Team projects on AWS — remote + locking
Azure Blob Storage	Team projects on Azure
Terraform Cloud	Enterprise, collaboration, UI dashboard
GCS (Google)	Team projects on GCP

03 HCL SYNTAX & BASICS

• Core HCL Syntax

```
# ── Provider Configuration ──────────────────────────
provider "aws" {
  region  = "us-east-1"
  profile = "default"
}

# ── Resource Block ──────────────────────────────────
resource "aws_instance" "web_server" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  tags = {
    Name        = "MyWebServer"
    Environment = "Production"
  }
}

# ── Variable Definition ─────────────────────────────
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

# ── Output Block ────────────────────────────────────
output "instance_ip" {
  value       = aws_instance.web_server.public_ip
  description = "Public IP of the web server"
}

# ── Data Source ─────────────────────────────────────
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]   # Canonical
```

```
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-*-22.04-amd64-server-*"]
  }
}
```

• Variable Types

Type	Example	Notes
string	type = string	Text values
number	type = number	Integer or float
bool	type = bool	true / false
list(string)	type = list(string)	Ordered list
map(string)	type = map(string)	Key-value pairs
object({...})	type = object({name=string})	Structured object
any	type = any	No type constraint

04 ESSENTIAL COMMANDS

• Setup & Initialization

```
# Install Terraform (Linux/Mac via tfswitch)
$ curl -L
https://raw.githubusercontent.com/warrensbox/terraform-switcher/release/install.sh | bash
$ tfswitch 1.7.0


# Initialize working directory (ALWAYS run first)
$ terraform init


# Initialize with upgrade providers
$ terraform init -upgrade


# Initialize with specific backend config
$ terraform init -backend-config=backend.hcl
```

• Plan & Apply Commands

```
# Preview what changes will be made
$ terraform plan


# Save plan to file for later apply
$ terraform plan -out=tfplan


# Apply changes (prompts for confirmation)
$ terraform apply


# Apply saved plan (no confirmation needed)
$ terraform apply tfplan


# Auto-approve without interactive prompt
$ terraform apply -auto-approve


# Target specific resource only
$ terraform apply -target=aws_instance.web_server


# Pass variable at command line
$ terraform apply -var="instance_type=t3.small"
```

```
# Use a tfvars file
$ terraform apply -var-file="prod.tfvars"
```

• Destroy Commands

```
# Destroy ALL managed infrastructure
$ terraform destroy

# Destroy with auto-approve
$ terraform destroy -auto-approve

# Destroy only specific resource
$ terraform destroy -target=aws_instance.web_server
```

04 COMMANDS (CONTINUED)

• Inspect & Debug Commands

```
# Show current state (all resources)
$ terraform show

# List all resources in state
$ terraform state list

# Show details of specific resource
$ terraform state show aws_instance.web_server

# Show output values
$ terraform output
$ terraform output instance_ip

# Format all .tf files (linting)
$ terraform fmt
$ terraform fmt -recursive     # includes subdirectories

# Validate configuration syntax
$ terraform validate

# Show dependency graph (DOT format)
$ terraform graph | dot -Tsvg > graph.svg
```

• State Management Commands

```
# Move resource in state (rename)
$ terraform state mv aws_instance.old aws_instance.new

# Remove resource from state (without destroying)
$ terraform state rm aws_instance.web_server

# Import existing resource into state
$ terraform import aws_instance.web_server i-1234567890abcdef0

# Pull remote state to local
```

```
$ terraform state pull

# Force unlock state (use with caution!)
$ terraform force-unlock LOCK_ID
```

• Workspace Commands

```
# List workspaces
$ terraform workspace list

# Create new workspace
$ terraform workspace new staging

# Switch workspace
$ terraform workspace select production

# Show current workspace
$ terraform workspace show

# Delete workspace
$ terraform workspace delete staging
```

04 COMMANDS QUICK REFERENCE

Command	Syntax	What It Does
init	terraform init	Initialize project, download providers
plan	terraform plan	Show execution plan (dry run)
apply	terraform apply	Execute plan, create resources
destroy	terraform destroy	Remove all managed resources
fmt	terraform fmt	Format .tf files automatically
validate	terraform validate	Check config for syntax errors
show	terraform show	Inspect current state or plan
output	terraform output	Display output values
state list	terraform state list	List all tracked resources
state show	terraform state show <res>	Detail a specific resource
state mv	terraform state mv a b	Rename resource in state
state rm	terraform state rm <res>	Remove from state only
import	terraform import <res> <id>	Bring existing resource under mgmt
taint	terraform taint <res>	Force recreation on next apply
untaint	terraform untaint <res>	Remove taint from resource
refresh	terraform refresh	Sync state with real infra
graph	terraform graph	Generate dependency graph (DOT)
providers	terraform providers	Show provider requirements
version	terraform version	Show Terraform version

05 PROJECT FILE STRUCTURE

• Recommended Project Layout

```
my-terraform-project/
├── main.tf          # Primary resource definitions
├── variables.tf     # Input variable declarations
├── outputs.tf       # Output value declarations
├── providers.tf     # Provider configuration
├── versions.tf      # Terraform & provider version constraints
├── terraform.tfvars # Default variable values (not in Git!)
├── backend.tf       # Remote state configuration
├── locals.tf        # Local value calculations
│
├── modules/         # Reusable modules
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── ec2/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
│
└── environments/    # Per-env variable files
    ├── dev.tfvars
    ├── staging.tfvars
    └── prod.tfvars
```

• versions.tf — Lock Provider Versions

```
terraform {
 required_version = ">= 1.5.0"

 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 5.0"
  }
  random = {
```

```
    source  = "hashicorp/random"
    version = "~> 3.5"
  }
 }
}
```

• Key Files to Add to .gitignore

Add these to .gitignore: .terraform/ terraform.tfstate terraform.tfstate.backup *.tfvars .terraform.lock.hcl (optional — some teams commit this)

06 MODULES & FUNCTIONS

• Using a Module

```
# Calling a local module
module "web_server" {
  source       = "./modules/ec2"
  instance_type = var.instance_type
  ami_id       = data.aws_ami.ubuntu.id
  env          = "production"
}

# Calling a registry module (Terraform Registry)
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["us-east-1a", "us-east-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]

  enable_nat_gateway = true
}
```

• Useful Built-in Functions

Function	Syntax	Purpose
length	length(list)	Count items in list/map
toset	toset(list)	Convert list to set
merge	merge(map1, map2)	Combine maps
lookup	lookup(map, key, default)	Safe map value lookup
join	join(",", list)	Join list into string
split	split(",", str)	Split string to list
format	format("Hello %s", var)	String formatting
upper/lower	upper(str)	Change string case
trimspace	trimspace(str)	Remove whitespace

coalesce	coalesce(a, b, c)	Return first non-empty value
file	file("path/to/file")	Read file contents
jsonencode	jsonencode(object)	Encode to JSON string
jsondecode	jsondecode(string)	Parse JSON string
base64encode	base64encode(string)	Base64 encode
cidrsubnet	cidrsubnet(cidr,8,1)	Calculate subnet CIDR

07 DEMO PROJECT: AWS EC2 WEB SERVER

Goal: Deploy a public EC2 instance running Nginx on AWS using Terraform — from scratch to running website. We'll provision: VPC, Subnet, Internet Gateway, Security Group, EC2 Instance.

• Project Structure

```
aws-webserver/
├── providers.tf
├── variables.tf
├── main.tf
└── outputs.tf
```

• providers.tf — Configure AWS Provider

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

• variables.tf — Input Variables

```
variable "aws_region" {
  description = "AWS region to deploy resources"
  type        = string
  default     = "us-east-1"
}

variable "project_name" {
  description = "Name tag for all resources"
  type        = string
```

```
  default     = "terraform-demo"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"        # Free-tier eligible
}

variable "allowed_ssh_cidr" {
  description = "CIDR allowed for SSH access"
  type        = string
  default     = "0.0.0.0/0"       # Restrict in production!
}
```

07 DEMO PROJECT (CONTINUED)

• main.tf — Core Infrastructure

```
# ── Data: Latest Ubuntu 22.04 AMI ──────────────────────────
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

# ── VPC ────────────────────────────────────────────────────
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "${var.project_name}-vpc" }
}

# ── Internet Gateway ───────────────────────────────────────
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${var.project_name}-igw" }
}

# ── Public Subnet ──────────────────────────────────────────
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "${var.aws_region}a"
  map_public_ip_on_launch = true
  tags = { Name = "${var.project_name}-public-subnet" }
}

# ── Route Table ────────────────────────────────────────────
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
```

```
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "${var.project_name}-rt" }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

07 DEMO PROJECT (CONTINUED)

• main.tf — Security Group & EC2 Instance

```
# ── Security Group ──────────────────────────────────────────
resource "aws_security_group" "web_sg" {
  name        = "${var.project_name}-sg"
  description = "Allow HTTP and SSH"
  vpc_id      = aws_vpc.main.id

  ingress {                    # Allow HTTP
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {                    # Allow SSH
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.allowed_ssh_cidr]
  }
  egress {                     # Allow all outbound
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = { Name = "${var.project_name}-sg" }
}

# ── EC2 Instance ────────────────────────────────────────────
resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = var.instance_type
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<-EOF
```

```
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    echo '<h1>Deployed with Terraform!</h1>' > /var/www/html/index.html
    systemctl start nginx
    systemctl enable nginx
  EOF

  tags = { Name = "${var.project_name}-web" }
}
```

• outputs.tf — Export Useful Values

```
output "instance_id" {
  value       = aws_instance.web.id
  description = "EC2 Instance ID"
}

output "public_ip" {
  value       = aws_instance.web.public_ip
  description = "Public IP — open in browser to see Nginx"
}

output "website_url" {
  value       = "http://${aws_instance.web.public_ip}"
  description = "Direct link to deployed website"
}
```

07 DEMO PROJECT — DEPLOY STEPS

• Step-by-Step Deployment

Step	Command & Description
1. Setup AWS CLI	aws configure — Enter Access Key, Secret, Region
2. Create Project Dir	mkdir aws-webserver && cd aws-webserver
3. Create .tf Files	Create providers.tf, variables.tf, main.tf, outputs.tf
4. Initialize	terraform init — Downloads AWS provider plugin
5. Validate	terraform validate — Check for syntax errors
6. Format	terraform fmt — Auto-format all .tf files
7. Plan	terraform plan — Review what will be created (5 resources)
8. Apply	terraform apply -auto-approve — Deploy to AWS
9. Access Site	Open http://<public_ip> in browser — see Nginx!
10. Cleanup	terraform destroy -auto-approve — Remove everything

• Expected terraform plan Output

```
Terraform will perform the following actions:

  # aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami                  = "ami-0c7217cdde317cfec"
      + instance_type        = "t2.micro"
      ...
    }

  # aws_security_group.web_sg will be created
  # aws_subnet.public will be created
  # aws_vpc.main will be created
  # aws_internet_gateway.igw will be created

Plan: 5 to add, 0 to change, 0 to destroy.
```

• Expected terraform apply Output

```
aws_vpc.main: Creating...
aws_vpc.main: Creation complete after 2s [id=vpc-0abc12345]
aws_internet_gateway.igw: Creating...
```

```
aws_subnet.public: Creating...
aws_security_group.web_sg: Creating...
aws_instance.web: Creating...
aws_instance.web: Still creating... [30s elapsed]
aws_instance.web: Creation complete after 45s [id=i-0abc12345]

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:
  public_ip   = "54.123.45.67"
  website_url = "http://54.123.45.67"
```

08 BEST PRACTICES & PRO TIPS

• Remote State with S3 Backend

```
# backend.tf — Store state remotely on S3 (team use)
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock"  # Prevent conflicts
  }
}
```

• Best Practices Summary

Practice	Why It Matters
Always run terraform plan first	Catch mistakes before real changes
Use remote state (S3/Terraform Cloud)	Team collaboration & no conflicts
Lock provider versions in versions.tf	Reproducible builds across environments
Never commit .tfstate or .tfvars to Git	Contains sensitive data & credentials
Use modules for repeated patterns	DRY principle — Don't Repeat Yourself
Use workspaces for environments	Separate dev/staging/prod with one codebase
Tag all resources	Cost tracking, compliance, and easy cleanup
Use data sources for existing infra	Avoid hardcoding IDs — stay dynamic
Enable state locking (DynamoDB)	Prevents simultaneous apply by team members
Use terraform fmt in CI/CD	Consistent code formatting in team projects

• Common Gotchas

Issue	Fix
Provider auth fails	Check env vars: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY

State drift	Run terraform refresh or terraform plan to detect
Resource already exists	Use terraform import to bring it under management
Plan shows -/+ (replace)	Immutable attribute changed — resource will be recreated
Lock error on apply	Someone else is running apply — wait or force-unlock
Module not found	Run terraform init to download/update modules

09 QUICK REFERENCE SUMMARY

• The 5 Most Used Commands

```
$ terraform init      # Step 1: Always run first
$ terraform fmt       # Step 2: Auto-format code
$ terraform validate  # Step 3: Check for errors
$ terraform plan      # Step 4: Preview changes
$ terraform apply     # Step 5: Deploy!
```

• HCL Block Types Quick Reference

Block Type	Purpose & Syntax
terraform {}	Core settings, version, backend, required_providers
provider "name" {}	Configure provider (auth, region, etc.)
resource "type" "name" {}	Define infrastructure to create/manage
data "type" "name" {}	Read existing infrastructure (read-only)
variable "name" {}	Declare input parameters
output "name" {}	Expose values after apply
locals {}	Define computed/local values
module "name" {}	Call a reusable module

• Terraform Lifecycle Summary

```
WRITE     INIT      PLAN      APPLY     DESTROY
  |         |         |         |         |
.tf files  Download   Show what   Execute   Remove all
in HCL     providers  will change  changes   resources
         & backend    (no changes  to cloud   from cloud
                      yet made)    infra      & state
```

🔗 **Written by AwaisUmarHayat | Follow for more on LinkedIn: linkedin.com/in/awaisumarhayat**